

Assessing the Viability of Machine Learning in Casting Defect Detection: A Comprehensive Analysis of Machine Learning Models

1st Arham Siddiqui

Santa Clara High School

Santa Clara, California

arham.siddiqui61@gmail.com

Abstract—Machine learning is the subfield of Artificial Intelligence in which a machine makes predictions from data. Depending on the type of data provided, algorithms under these categories will vary in accuracy and efficiency when applied. Casting is a manufacturing process where liquid material is poured into a mold containing a hollow cavity of the desired shape. This liquid material later solidifies in the shape of the mold. However, in the molding and casting industry, defects in the casting material such as blow holes, pinholes, burr, shrinkage defects, mold material defects, pouring metal defects, and metallurgical defects call for a manual, time consuming, and inaccurate quality inspection process. In this paper, we show that the application of machine learning in the molding and casting industry can serve to be an accurate, more cost effective, and timely alternative to the current quality inspection process. Multiple machine learning methods are applied, evaluated, and analyzed thoroughly to reach a conclusive statement. Our results are then compared to those of recent advancements in surface defect detection.

I. INTRODUCTION

Manufacturing products in the industrial process inevitably leads to defects. Major reasons for the cause of these defects include faulty design and machine production equipment, unfavorable working conditions, and fragile material. If left untreated, manufacturing defects can induce a spiral of several problems in the workplace: increased production cost, shortened service life of products, extensive consumption of resources. Manual solutions to this problem are costly, inaccurate, and time-insensitive. Alternatively, automatic, or machine-based solutions, have proved to be superior in these aspects of detecting defective products [8].

Due to the pronounced necessity for automated detection systems, there has been extensive research conducted around this field. For the simplicity of this paper and to better direct the goals of this research, we will be taking a look at a specific industrial manufacturing process which is prone to defects: molding and casting. Casting is a manufacturing process where liquid material is poured into a mold containing a hollow cavity of the desired shape. This liquid material later solidifies into the shape of the mold. Defects in this industry most commonly include imperfections in the casting material, more specifically blow holes, pinholes, burr, shrinkage defects, mold material defects, pouring metal defects, and metallurgical defects [2].

The defects in casting material previously mentioned are present on the surface area of the cast. The technology used to detect defects in this area can be further defined as the technology used to detect spots, pits, scratches, and color differences on a product's surface. As for detecting these imperfections, there are various automated procedures currently in place. These include deep learning [7], magnetic powder [4], eddy current testing [3], ultrasonic testing [6], and machine vision [5] detection methods [8].

The contributions of this paper include: Testing and analyzing common machine learning models to identify the most accurate model in surface imperfection detection, calibrating a deep learning model to give us the best performance when detecting defects on casting material, evaluating the performance of this deep learning model to current novel detection processes in regards to detection accuracy, understanding how our adjustments to the deep learning model increased the accuracy of the detection, and finding a correlation between the adjustments made and the model's performance.

This paper is organized as the following: In the first section, issues caused by casting imperfections are established and Deep Learning defect detection is introduced. In the second section, our methods and procedures are demonstrated and explored. In the third section, our results are presented and evaluation metrics are explained. In the fourth section, these results are thoroughly analyzed. The last section concludes our findings.

II. METHODS

Dataset

The data we used was obtained from kaggle.com [2]: Casting Product Image Data For Quality Inspection. The dataset contains 7,348 300x300x3 images of defective and non-defective casting material. After importing the dataset to a local notebook, we converted all the images from rgb to grayscale and shuffled the images in both the training and testing alongside their corresponding labels to avoid any similarity bias whatsoever.

Models

The first model we applied the data to was Logistic Regression. To do so, we first flattened all the images and fit the

data to the model with a max iteration of 200. Additionally, we applied the flattened data to the Random Forest Classifier. Both of these models are provided by the scikit-learn package. We evaluated the performance of both of these models by using a confusion matrix.

Since image classification is a problem of linear topology, we applied the non-flattened data to a Sequential model provided by the tensorflow package. Initially, we added 3 convolutional and maxpool layers with a Leaky Relu activation. This architecture can be seen in Fig 1.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 298, 298, 8)	80
leaky_re_lu (LeakyReLU)	(None, 298, 298, 8)	0
max_pooling2d (MaxPooling2D)	(None, 149, 149, 8)	0
conv2d_1 (Conv2D)	(None, 147, 147, 8)	584
leaky_re_lu_1 (LeakyReLU)	(None, 147, 147, 8)	0
max_pooling2d_1 (MaxPooling2D)	(None, 73, 73, 8)	0
conv2d_2 (Conv2D)	(None, 71, 71, 8)	584
leaky_re_lu_2 (LeakyReLU)	(None, 71, 71, 8)	0
max_pooling2d_2 (MaxPooling2D)	(None, 35, 35, 8)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 8)	0
dense (Dense)	(None, 1)	9
Total params: 1,257		
Trainable params: 1,257		
Non-trainable params: 0		

Fig. 1.

An additional convolutional and maxpool layer was added, with a Leaky Relu activation, to assess the resulting accuracy. Both Sequential models were compiled with the Adam optimizer along with Binary Crossentropy for the loss and were run for 10 epochs. The architecture for the 4 layer model can be seen in Fig 2.

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 298, 298, 8)	80
leaky_re_lu_7 (LeakyReLU)	(None, 298, 298, 8)	0
max_pooling2d_7 (MaxPooling2D)	(None, 149, 149, 8)	0
conv2d_8 (Conv2D)	(None, 147, 147, 8)	584
leaky_re_lu_8 (LeakyReLU)	(None, 147, 147, 8)	0
max_pooling2d_8 (MaxPooling2D)	(None, 73, 73, 8)	0
conv2d_9 (Conv2D)	(None, 71, 71, 8)	584
leaky_re_lu_9 (LeakyReLU)	(None, 71, 71, 8)	0
max_pooling2d_9 (MaxPooling2D)	(None, 35, 35, 8)	0
conv2d_10 (Conv2D)	(None, 33, 33, 8)	584
leaky_re_lu_10 (LeakyReLU)	(None, 33, 33, 8)	0
max_pooling2d_10 (MaxPooling2D)	(None, 16, 16, 8)	0
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 8)	0
dense_2 (Dense)	(None, 1)	9
Total params: 1,841		
Trainable params: 1,841		
Non-trainable params: 0		

Fig. 2.

III. RESULTS

Evaluation Metrics

To compare the performance of the Logistic Regression and RFC models, we used the confusion matrix (prediction vs actual), AUC plot (area under ROC plot (plots TPR vs. FPR at different classification thresholds [1])), and recall score (TP/TP+FN).

To compare the performance of the Sequential Architectures, we visualized the accuracy and loss scores over epochs and plotted respective confusion matrices for the validation accuracy.

Results

The Logistic Regression model had an AUC score of 89.1% and a recall of 82.4%. This model performed decently though all the data given was flattened. The confusion matrix for this model is shown in Fig 3 and the AUC score is visualized in Fig 4.

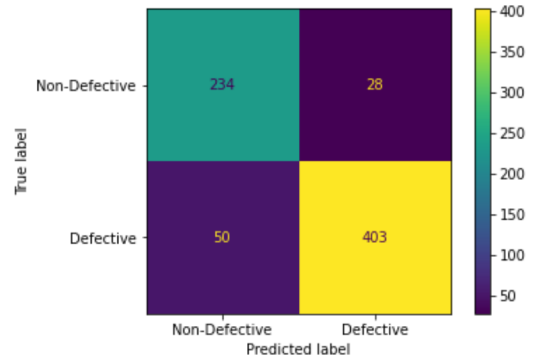


Fig. 3.

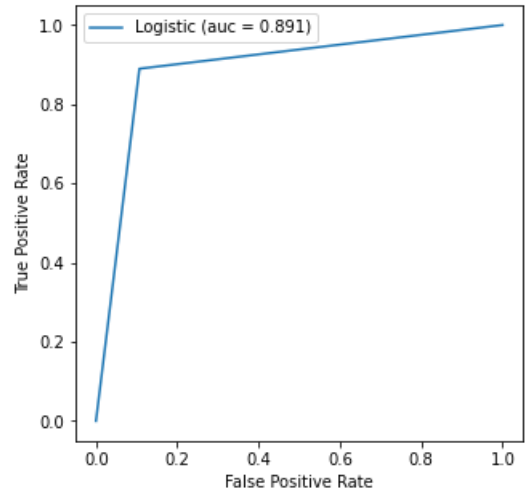


Fig. 4.

The Random Forest Classifier had an AUC score of 98.2% and a recall of 96.6%. This model performed extremely well due to the complexity of the numerous trees and the amount of

verification behind each prediction. The confusion matrix for this model is shown in Fig 5 and the AUC score is visualized in Fig 6.

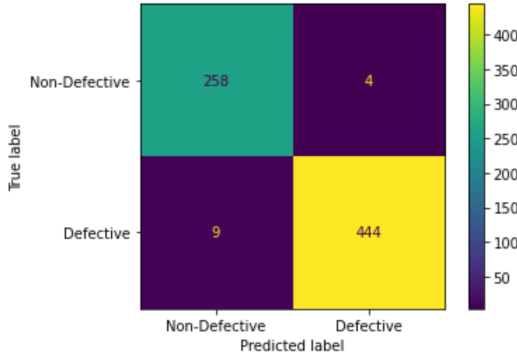


Fig. 5.

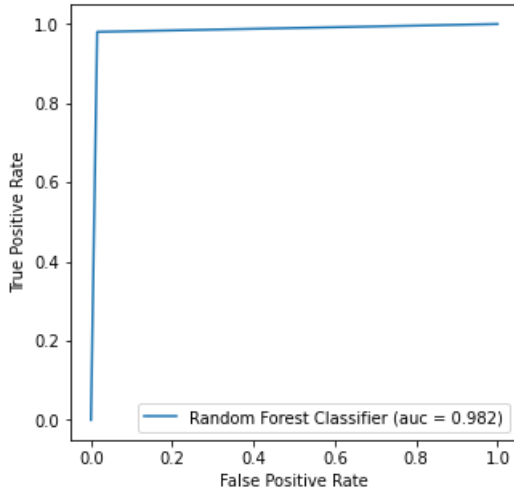


Fig. 6.

The 3 layered Sequential CNN had an accuracy of 79.0%. There is a steep decrease in loss during epochs 0-4 after which the model stabilizes. The confusion matrix of this model is shown in Fig 7. The accuracy and loss plots are visualized in Fig 8.

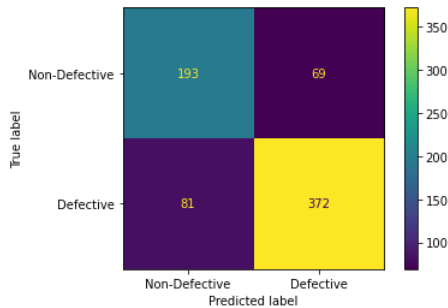


Fig. 7.

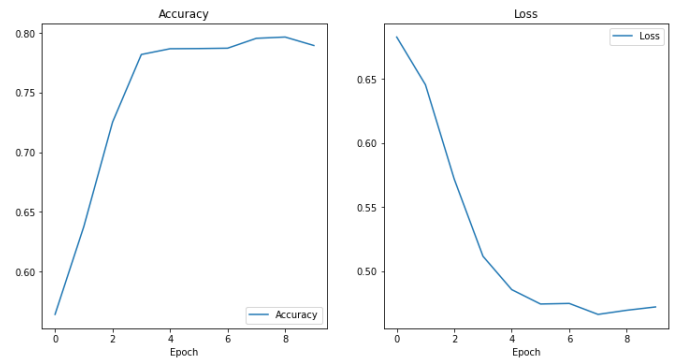


Fig. 8.

The 4 layered Sequential CNN had an accuracy of 82.5%. The loss curve exhibits stabilization after epoch 5-6. The confusion matrix of this model is shown in Fig 9. The accuracy and loss plots are visualized in Fig 10.

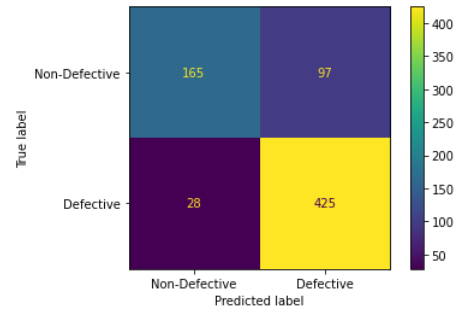


Fig. 9.

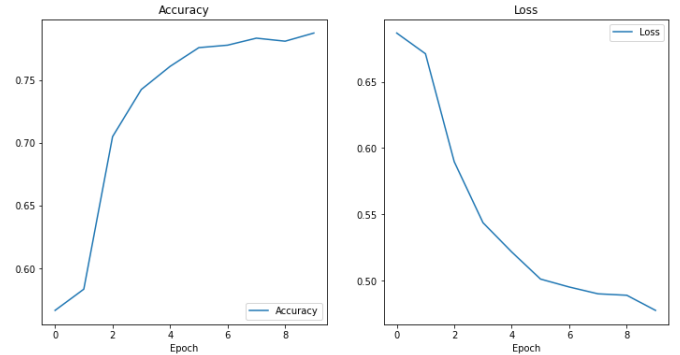


Fig. 10.

IV. ANALYSIS

The evaluation of these four methods will be broken down into a comparison between the Logistic Regression and Random Forest Classifier, both Sequential architectures, and a final comparison between all 4 proposed methods.

Logistic Regression vs Random Forest Classifier

Analyzing the AUC scores of the Logistic Regression model and the RFC, 89.1% and 98.2% respectively, the difference of

9.1% between these scores indicates an apparent superiority of the RFC over the Logistic Regression model regarding the ability of the model to distinguish between positive and negative classes in the testing data. With a difference of 14.2% in the recall score, the RFC definitively outclasses the Logistic Regression model in its ability to repeatedly output positive predictions. As we have both confusion matrices visualized for these models, we can delve further into performance analysis by utilizing the precision score. As precision is calculated by dividing the number of True Positive values by the total of the True Positives and False Positives, we must define these terms. Defective cases will be classified as positive while non-defective cases will be classified as negative. Accordingly, the precision score for the Logistic Regression model is 93.5% and for the Random Forest Classifier it is 99.1%. This metric shows us that there is a far lower margin of error regarding positive classifications in the Random Forest Classifier as opposed to the Logistic Regression model. Overall, the RFC proved superiority over the Logistic Regression model after evaluating the AUC, recall, and precision metrics.

Sequential Architectures

Analyzing the accuracy scores of both architectures, with the 3 layer having 79.0% and the 4 layer having 82.5%, it can be concluded that the 4 layer architecture does perform better. Evaluating the precision scores, which are calculated from the respective visualized confusion matrices, the 3 layer model has 84.4% precision level while the 4 layer has 81.4%. This strikes interest as the higher accuracy in the 4 layer model doesn't translate to superior precision score over that of the 3 layer model. This may be because precision is not affected by the amount of layers in the architecture. This suggests that the 4 layer model may have a higher tendency to produce false positives or negatives compared to the 3 layer model. On the other hand, the 4 layer model has about a 15% increase in specificity, which regards the amount of true negative predictions. This disparity is more significant than the mere 3% variation between the recall score of the two models. Therefore, greater emphasis should be placed on this metric when evaluating overall model performance. Overall, adding an additional layer to the 3 layered Sequential CNN proved beneficial as there was increase in accuracy and specificity, whereas a slight deterioration in precision level. The decreased precision level may have been a result of increased complexity in the model, leading to a lack of generalization and/or feature extraction from the data.

All 4

Comparing the accuracy scores of all 4 proposed models, the RFC and Logistic Regression models both outperformed the Sequential architectures. These "simple" models likely outperformed the Sequential models because they were better suited for the task at hand, specifically the binary classification of this particular dataset. The size of the dataset plays a role in the accuracy of the Sequential models as they are typically better suited for larger datasets to generate more

accurate generalizations. Additionally, an increase in accuracy was observed from the 3 layer to 4 layer model, indicating a potential correlation between the number of model parameters and the resulting accuracy score.

V. CONCLUSION

It can be concluded that the Random Forest Classifier outperformed all other proposed methods in this study. This model's ensemble architecture and robustness to outliers and noise makes it ideal for this case of binary classification. Based on the accuracy scores observed and our knowledge of deep learning models, we can conclude that the Sequential architectures proposed would need to be increased in size (total parameters) and applied to a larger dataset to rival the accuracy shown by the RFC.

On this note, the question arises why a larger dataset wasn't applied to this case. Rotational data augmentation was performed to this data set to quadruple the amount of data values available for training and testing. Unfortunately, the augmented data was unable to be utilized due to insufficient memory on the local computing device responsible for training and testing the models. To combat this issue, data streaming by the use of generators was explored but was unsuccessful in addressing the memory limitations. Regardless, we can hypothesize an increase in the Sequential models' accuracy if they were to be applied on the augmented dataset if more layers were added to their architecture.

The 98.2% AUC score of the RFC is sufficient for this case. The Random Forest Classifier outperforms all other machine learning models tested on an artificial and real defect dataset proposed by Yin et al, where amplitude, width, angle, and symmetry were computed to extract features from Lissajous figures [3]. Additionally, there is minimal cost associated with a machine learning implementation in a real life setting: camera, a computing device and an accompanying alert system. These costs outweigh the alternative consequences of undetected casting defects or hiring human labor for this cause.

REFERENCES

- [1] "Classification: Roc Curve and AUC ; — ; Machine Learning ; — ; Google for Developers." Google, developers.google.com/machine-learning/crash-course/classification/roc-and-auc. Accessed 14 July 2023.
- [2] Dabhi, Ravirajsinh. "Casting Product Image Data for Quality Inspection." Kaggle, 2 July 2020, www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product.
- [3] D'Angelo, Gianni et al. "Fast Eddy Current Testing Defect Classification Using Lissajous Figures." IEEE Transactions on Instrumentation and Measurement 67 (2018): 821-830.
- [4] Influence of Si/Cr Ratio on Flake Morphology and Magnetic Properties of Fe-Si-Al-Cr Particles. Zhang, N., +, LMAG 2016 3706005
- [5] Jing, Yang & Li, Shaobo & Gao, Zong & Wang, Zheng & Liu, Wei. (2018). Real-Time Recognition Method for 0.8 cm Darning Needles and KR22 Bearings Based on Convolution Neural Networks and Data Increase. Applied Sciences. 8. 1856-1873. 10.3390/app8101857.
- [6] Kusano, Masahiro et al. "Mid-infrared pulsed laser ultrasonic testing for carbon fiber reinforced plastics." Ultrasonics vol. 84 (2018): 310-318. doi:10.1016/j.ultras.2017.11.015
- [7] Y. Li, W. Zhao and J. Pan, "Deformable Patterned Fabric Defect Detection With Fisher Criterion-Based Deep Learning," in IEEE Transactions on Automation Science and Engineering, vol. 14, no. 2, pp. 1256-1264, April 2017, doi: 10.1109/TASE.2016.2520955.

- [8] Yang, J.; Li, S.; Wang, Z.; Dong, H.; Wang, J.; Tang, S. Using Deep Learning to Detect Defects in Manufacturing: A Comprehensive Survey and Current Challenges. *Materials* 2020, 13, 5755. <https://doi.org/10.3390/ma13245755>
- [9] Yin L, Ye B, Zhang Z, Tao Y, Xu H, Salas Avila JR, Yin W, A novel feature extraction method of eddy current testing for defect detection based on machine learning, *NDT and E International* (2019), doi: <https://doi.org/10.1016/j.ndteint.2019.04.005>.